

PANGASINAN HERITAGE DIGITAL SHOWCASE

Activity 1.1 — Framework Selection Report

Project: The Pangasinan Heritage Digital Showcase

The brief requires a platform that is lightning fast on 3G/4G, mobile-first, maintainable, accessible to WCAG 2.1 AA principles, and suitable for static/JAMstack-style deployment. It specifies two choices: React with Next.js 14 (App Router), or Vue with Nuxt.js 3.

Decision: Next.js 14 (App Router)

Next.js 14 is selected because it provides a strong fit for the requested React option, with an App Router architecture that supports server-rendered UI, reusable components, and static generation. The project can keep interactive behavior small while producing a fast mobile-first site.

Criterion	Next.js 14	Nuxt.js 3
Bundle / performance	5/5	4/5
Developer velocity	5/5	4/5
Ecosystem maturity	5/5	4/5
Learning curve	4/5	4/5
Component architecture	5/5	5/5
Docs/community	5/5	4/5
Project suitability	5/5	4/5
Weighted total	34/35 (97%)	29/35 (83%)

Quantitative method

Each criterion is scored from 1 (weak fit) to 5 (excellent fit), with equal weighting. The scores are a project-fit assessment based on the requirements in the activity brief, not benchmark measurements. This makes the recommendation transparent without claiming unsupported exact bundle-size numbers.

Bundle size and performance

Next.js receives 5/5 because the selected architecture supports server-side rendering and static generation, allowing the showcase to minimize unnecessary client-side JavaScript. Images can be lazy-loaded and responsive layouts can prioritize small screens. Nuxt is also strong, but Next.js has the stronger fit for this selected React-based submission.

Developer velocity and ecosystem

Next.js receives 5/5 for the availability of reusable React components and a mature ecosystem. Nuxt receives 4/5: it is also mature and component-oriented, but the assignment's selected implementation is React/Next.js.

Learning curve

Both receive 4/5. Both frameworks add conventions beyond their underlying UI libraries. Next.js concepts such as server/client components and App Router require deliberate learning, while Nuxt introduces its own conventions.

Component architecture and accessibility

Both support modular components. This implementation uses Atomic Design to keep atoms, molecules, and organisms reusable. Accessibility is treated as an implementation responsibility: semantic HTML, labels, keyboard-friendly controls, responsive layouts, and descriptive text are included.

Conclusion

Next.js 14 (App Router) is the recommended framework for this project. Its architecture aligns well with a fast, maintainable, component-based showcase and a static/JAMstack-style deployment strategy. The implementation deliberately avoids unnecessary dependencies and keeps the page content lightweight for mobile users.